

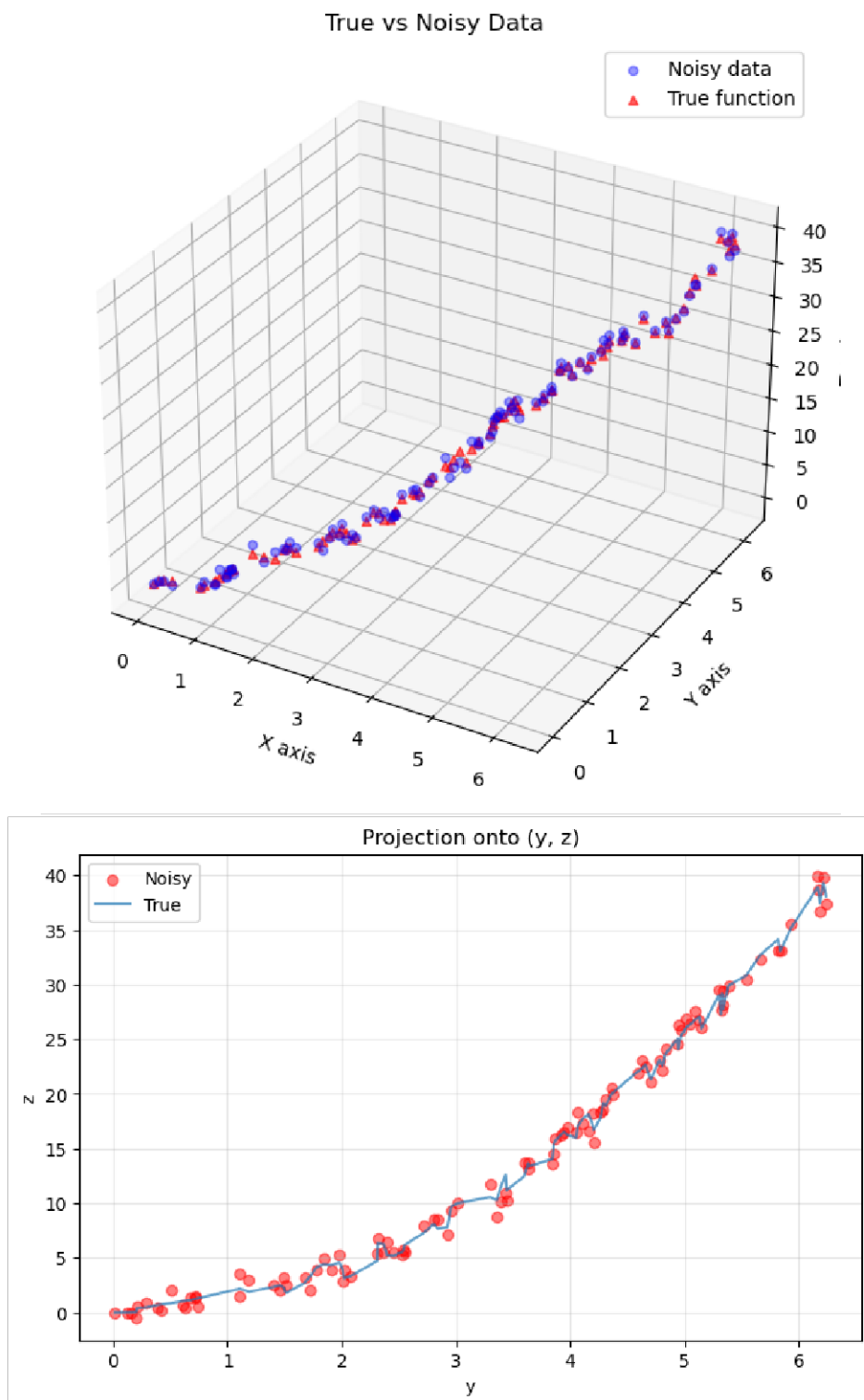
Assignment 1 (AM5920) - Kanak Agarwal (ID25M802)

Dataset Description

The function used to generate the dataset is,

$$f(x, y) = \sin(x^3) + y^2$$

Noise was added to the dataset by sampling from a normal distribution with mean 0 and standard deviation 0.7. The dataset consists of 100 samples, where each sample is a pair of input features (x, y) and a corresponding target value $f(x, y) + \text{noise}$. The input features x and y were uniformly sampled from the range $[0, 2\pi]$.



Question 1 - Linear Regression

Implementation (Algorithm)

Input:

- Feature vectors: x, y - Target values: z - Learning rate: η - Number of epochs: N

Output:

- Trained linear regression model

Steps

1. Prepare Training Data

- Stack input features to form

$$X = [x, y]$$

- Convert X and z to tensors

2. Initialize Model

- Define a linear model

$$z = W_1x + W_2y + b$$

- Initialize weights and biases

3. Select Device

- Use GPU if available, otherwise CPU
- Move model and data to selected device

4. Define Optimization Components

- Loss function: Mean Squared Error (MSE)
- Optimizer: Stochastic Gradient Descent (SGD) or Adam Optimizer

5. Training Loop

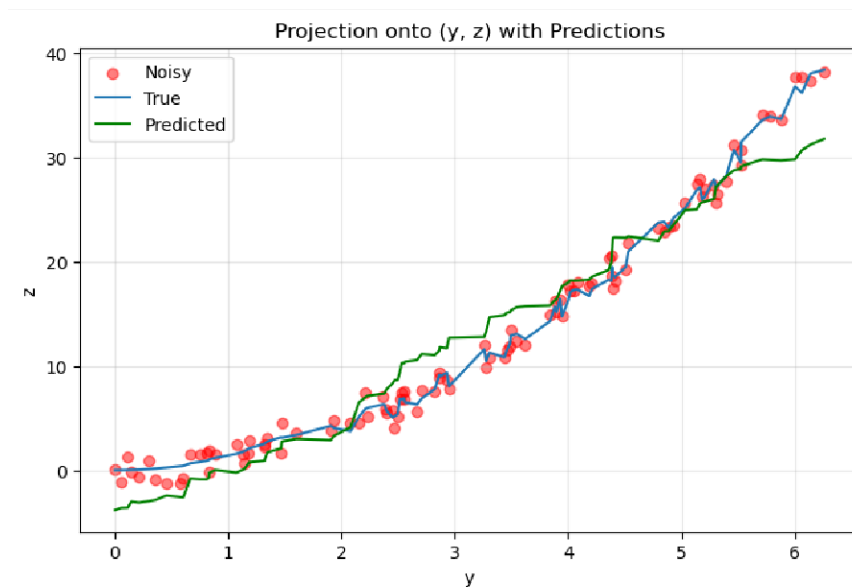
- For epoch = 1 to N :
 - Perform forward pass to compute predictions
 - Compute loss between predictions and targets
 - Backpropagate gradients
 - Update model parameters
 - Record training loss

6. Return

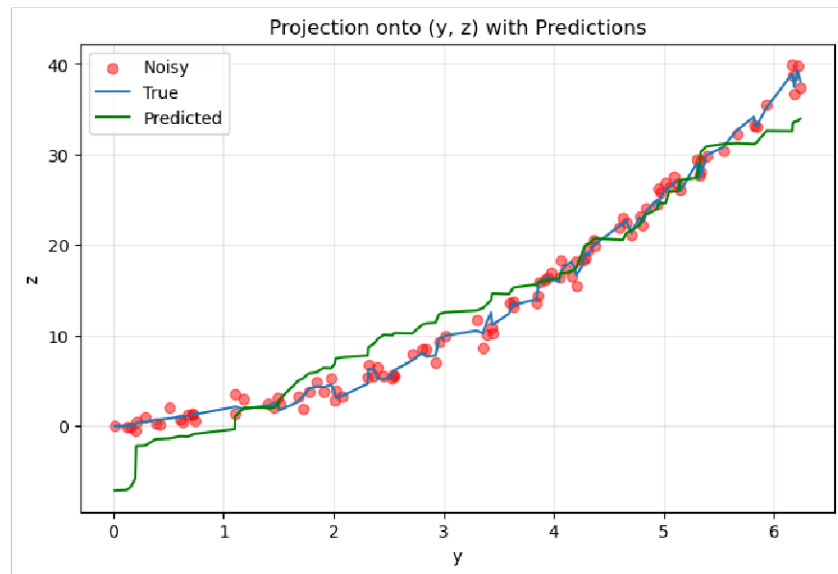
- Trained model parameters

Results

Stochastic Gradient Descent (SGD)

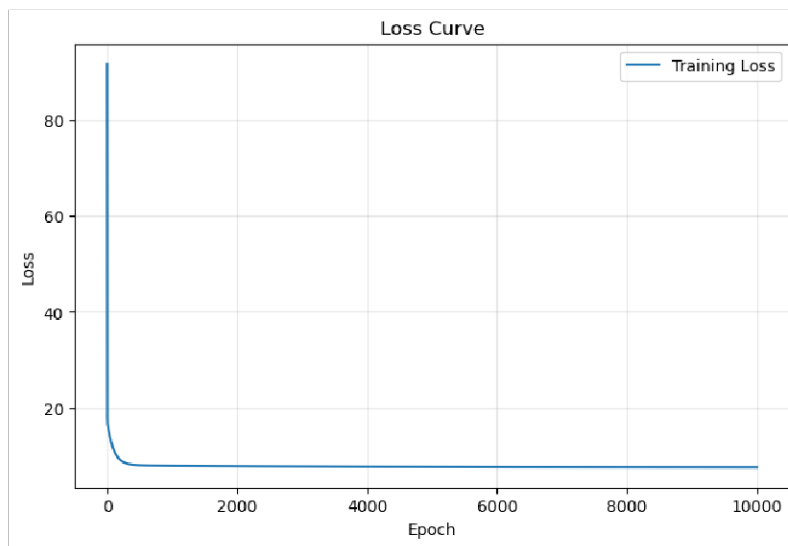


Adam Optimizer

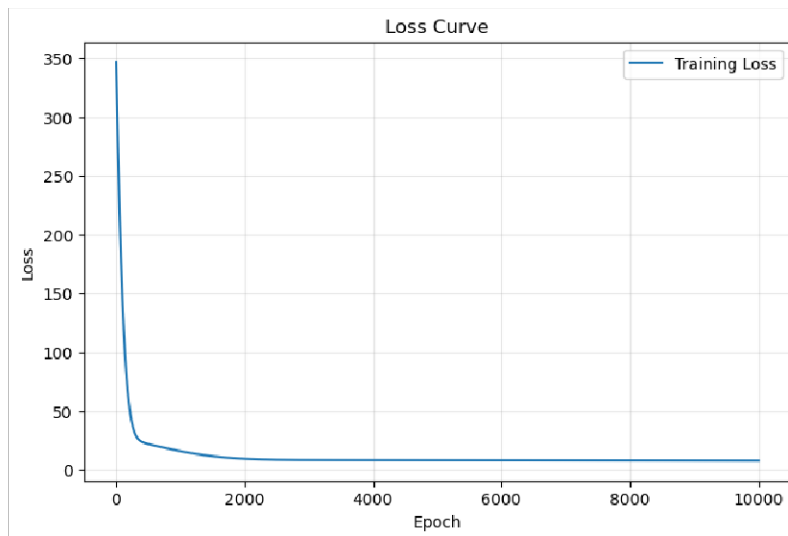


Final loss: 7.740602970123291

Convergence Plots



Stochastic Gradient Descent (SGD)



Adam Optimizer

Discussion of the Results

The linear regression model, both with SGD and Adam optimizers, achieved a final loss of around 7.6 and 7.7, indicating that it was able to capture some of the underlying structure of the data but is limited by its linear nature. The presence of noise and the non-linear relationship between the features and the target variable contribute to the relatively high loss, as a linear model cannot accurately capture the complexity of this particular highly non-linear function.

For the same learning rate, it is observed that SGD converged faster when compared to the Adam optimizer. This is the case for this particular dataset and model, but this doesn't reflect the general performance of these optimizers across all problems. Adam is typically more effective for training deep neural networks, while SGD can perform well for simpler models.

Question 2

Implementation (Algorithm)

Input:

- Feature vectors: x, y
- Target values: z
- Hidden layer sizes: $[h_1, h_2, \dots, h_L]$
- Activation function: $\sigma(\cdot)$
- Learning rate: η
- Number of epochs: N

Output:

- Trained neural network model

Steps

1. Prepare Training Data

- Stack input features to form

$$X = [x, y]$$

- Convert X and z into tensors

2. Initialize Neural Network

- Create a feedforward network with:
 - Input layer of size 2
 - L hidden layers with neurons $[h_1, h_2, \dots, h_L]$
 - Activation function $\sigma(\cdot)$ after each hidden layer
 - Output layer of size 1

3. Select Computational Device

- Use GPU if available, otherwise CPU
- Move model and data to the selected device

4. Define Training Components

- Loss function: Mean Squared Error (MSE)
- Optimizer: Adam

5. Training Loop

- For epoch = 1 to N :
 1. Perform forward pass to compute predictions

$$\hat{z} = f_{\theta}(X)$$

2. Compute loss

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (\hat{z}_i - z_i)^2$$

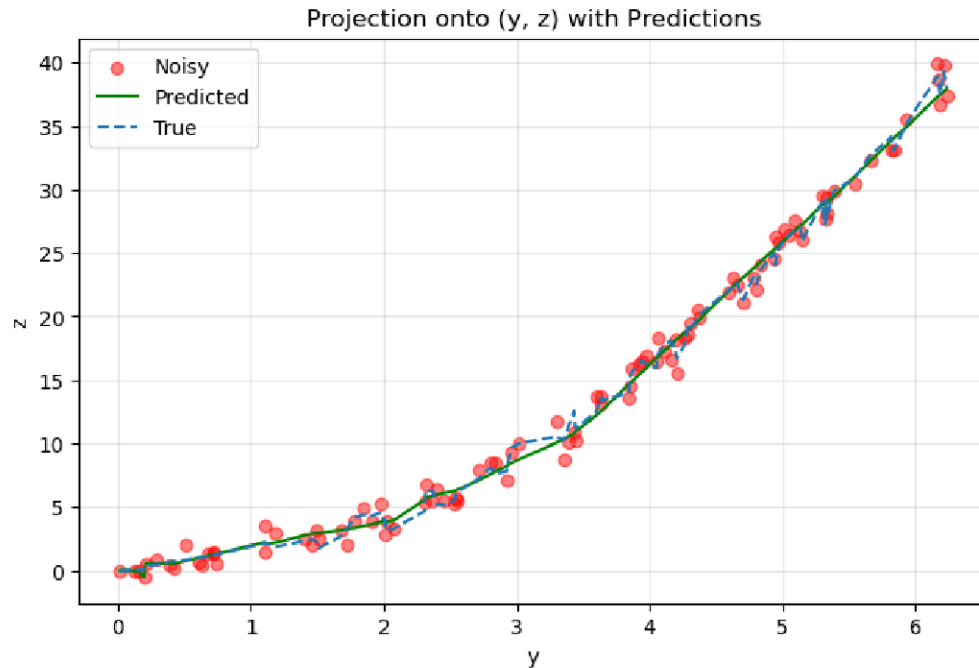
3. Backpropagate gradients
4. Update network parameters
5. Store training loss

6. Return

- Trained neural network parameters

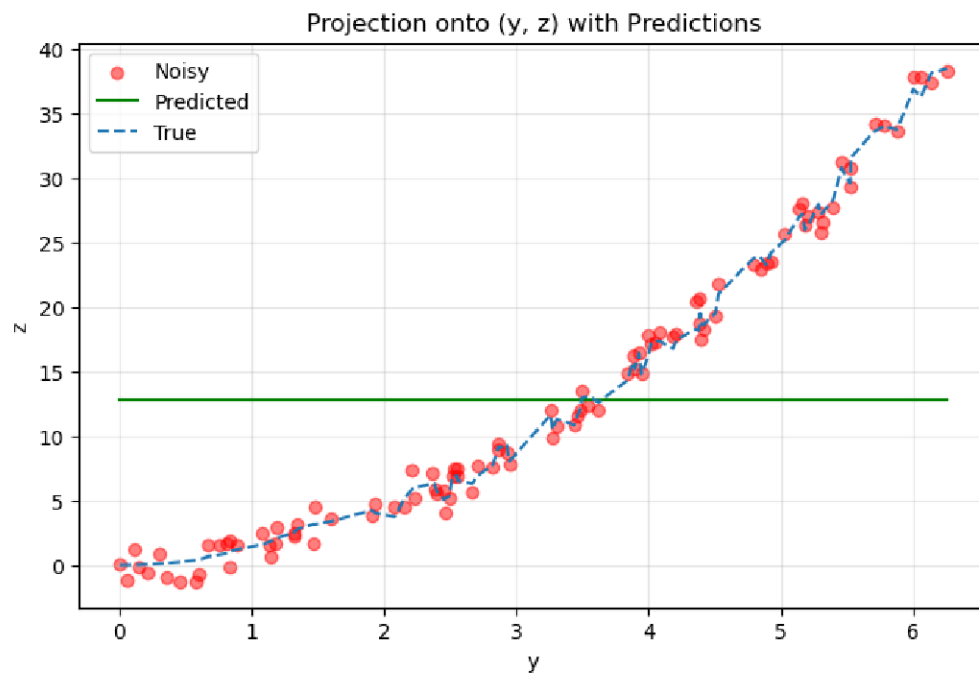
Results

Neural Network with 2 hidden layers and 10 neurons each



Final Loss: 0.8195960521697998

Neural Network with 10 hidden layers and 2 neurons each



Final Loss: 122.52734375

Discussion of the Results

The neural network with 2 hidden layers and 10 neurons each performed significantly better, achieving a much lower final loss of approximately 0.82, suggesting that it was able to model the non-linear relationship in the data effectively. In contrast, the neural network with 10 hidden layers and only 2 neurons each performed poorly, with a final loss of around 122.5. This occurs due to insufficient capacity to learn the complex function given the small number of neurons in each layer. This further cements the importance of choosing the right architecture for the problem at hand, as too few neurons can lead to underfitting, while too many layers with insufficient neurons can also hinder learning.

Question 3

Implementation (Algorithm)

Algorithm: Training a Residual Neural Network for Regression

Input:

- Feature vectors: x, y
- Target values: z
- Number of residual blocks: B
- Hidden layer width: h
- Activation function: $\sigma(\cdot)$
- Learning rate: η
- Number of epochs: N

Output:

- Trained residual neural network model

Steps

1. Prepare Training Data

- Stack input features to form

$$X = [x, y]$$

- Convert X and z to tensors

2. Initialize Residual Network

- For $b = 1$ to B :
 - Define a residual block

$$r_b(x) = x + W_{b,2} \sigma(W_{b,1}x + b_{b,1}) + b_{b,2}$$

- Append a final linear output layer

3. Select Computational Device

- Use GPU if available, otherwise CPU
- Move model and data to selected device

4. Define Training Components

- Loss function: Mean Squared Error (MSE)
- Optimizer: Adam

5. Training Loop

- For epoch = 1 to N :
 1. Forward propagate through residual blocks

$$\tilde{x}_{b+1} = r_b(\tilde{x}_b)$$

2. Compute network output

$$\hat{z} = W_{\text{out}}\tilde{x}_B + b_{\text{out}}$$

3. Compute loss

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (\hat{z}_i - z_i)^2$$

4. Backpropagate gradients

5. Update parameters

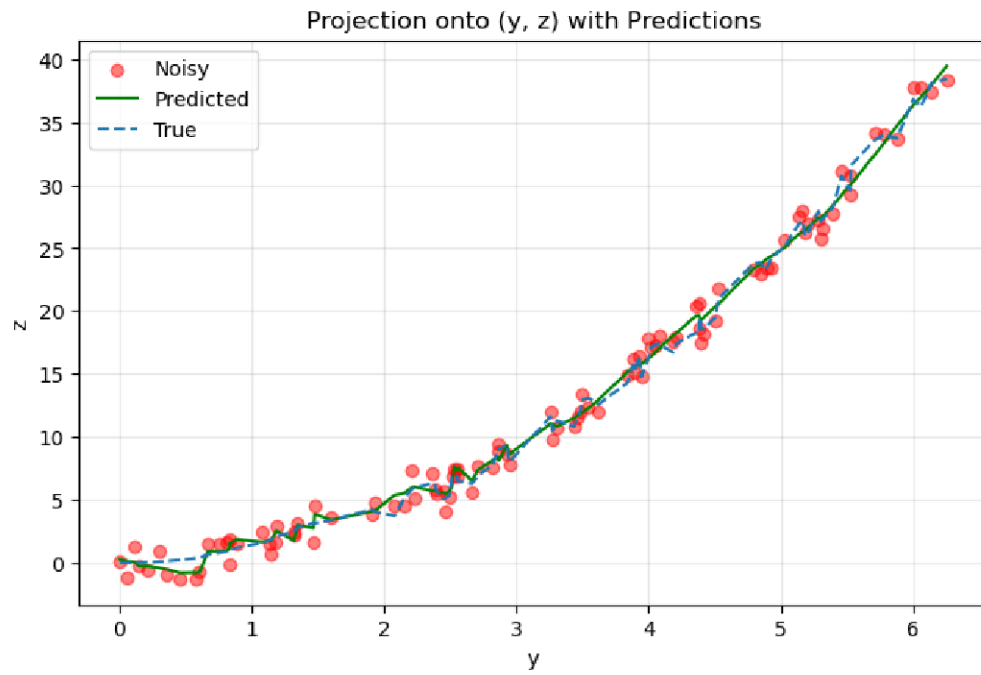
6. Return

- Trained residual network parameters

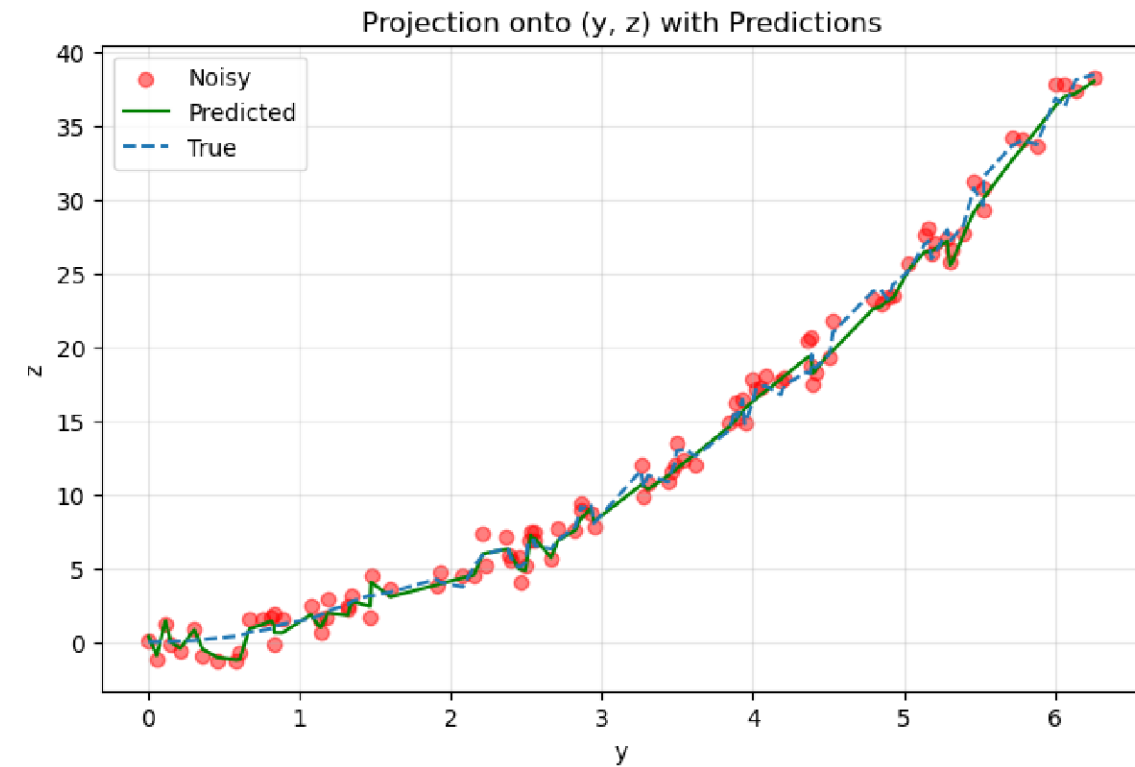
Results

The width of the non-linear correction map (analogous to width) is varied and the number of residual blocks (analogous to depth) is varied to observe the effect on the final loss.

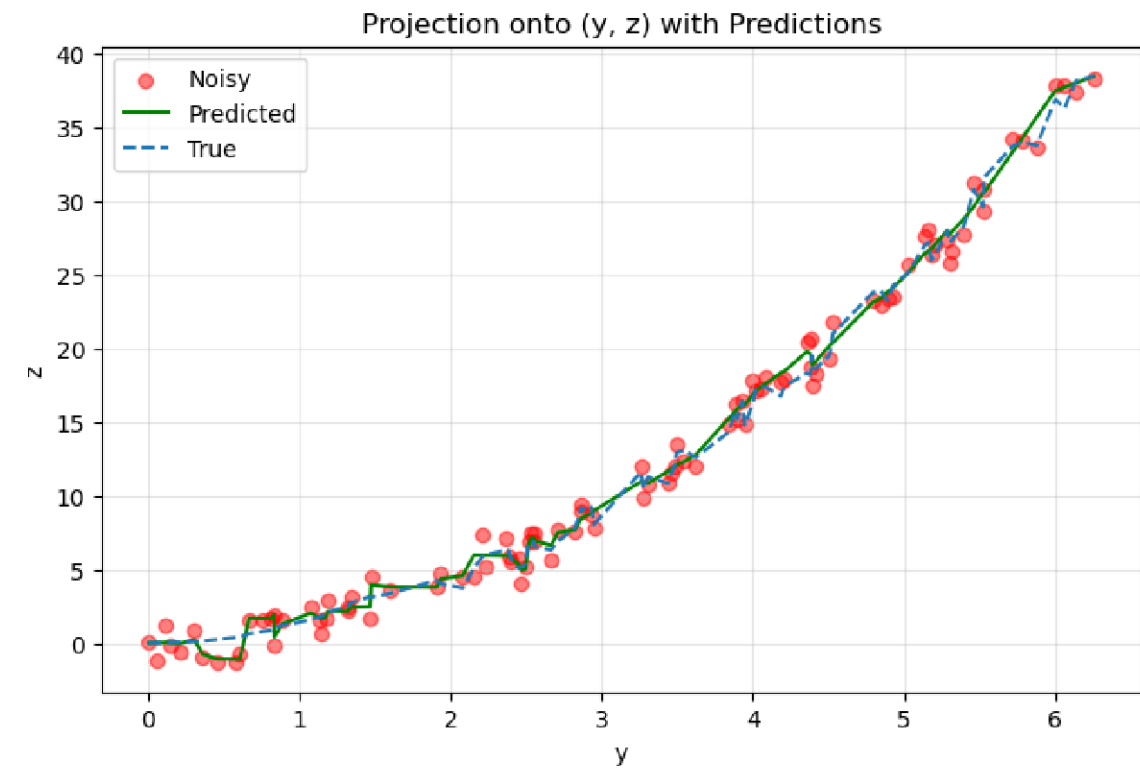
Residual Network with width 10 and depth 3



Residual Network with width 100 and depth 3



Residual Network with width 3 and depth 100



Discussion of the Results

The residual network with a width of 100 and depth 3, clearly overfitted the data, losing its generalization capabilities. A similar trend was observed with the network with width 3 and depth 100, this particular model also took considerably longer to train. The network with a width of 10 and depth 3 performed significantly better, although it still overfits in certain regions.